

AI-NATIVE ENGINEERING WITH CLAUDE CODE

From Business Specification to a Model-Driven Customer Onboarding System

Initial Engineering Foundation • Claude Code • CLAUDE.md • spec.md • Project Harness

Purpose of This Document

This document captures the initial engineering discussion for building an enterprise Customer Onboarding AI application using Claude Code from the terminal. The first part deliberately starts with specification-driven software engineering: business problem → specification → project instructions → architecture → plan → tasks → implementation. The later sections show why this foundation must evolve when the application itself becomes model-driven and probabilistic.

Important Terminology

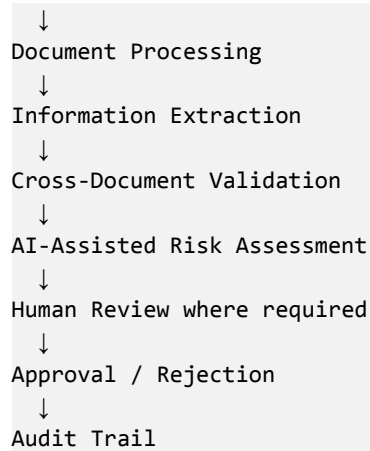
- CLAUDE.md — persistent project guidance and context for Claude Code, such as architecture, conventions, workflows and commands. It is guidance/context, not a hard enforcement mechanism.
- spec.md — the project's functional and non-functional specification. It is an engineering convention, not a special Claude Code filename.
- Claude Code — Anthropic's coding agent used from the terminal to inspect, modify, test and reason over the repository.
- Project harness — the surrounding repository structure, instructions, rules, tools, state and verification mechanisms that make agentic development more reliable.
- AI-native engineering — engineering software in which probabilistic models are first-class components, requiring model contracts, evaluation, uncertainty handling and human escalation.
- Harness engineering — the broader practice of designing the environment and feedback mechanisms around an agent so it can perform complex, iterative, long-running work reliably.

1. The Business Problem: Enterprise Customer Onboarding

The running example is a corporate customer onboarding process. A customer submits an application and supporting documents. The organization must extract information, validate consistency, assess risk, make a decision, retain evidence and maintain an auditable record.

Target Workflow

```
Customer
↓
Application / Registration
↓
Document Upload
```

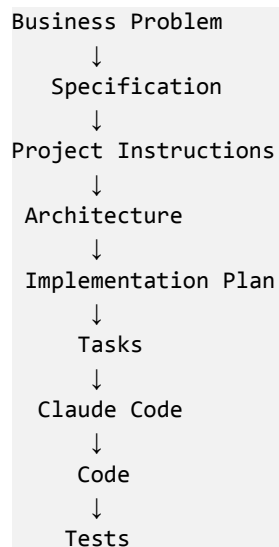


Core Business Capabilities

- Corporate customer registration
- Document upload and processing
- Information extraction
- Cross-document discrepancy detection
- AI-assisted risk assessment
- Human review and escalation
- Approval/rejection workflow
- Evidence and auditability

2. Why Start with Specification-Driven Engineering?

Before asking Claude Code to write application code, the engineering team should establish what is being built, why it is being built, what constraints apply and how success will be verified. This prevents the coding agent from inventing business requirements and creates a durable context layer that can be reused across multiple coding sessions.



3. CLAUDE.md — How Claude Should Work

CLAUDE.md should describe the project context and working conventions Claude Code should follow. It can contain the project purpose, architecture principles, technology choices, coding standards, testing commands, security expectations and development workflow.

Recommended content:

- Project purpose and business context
- Technology stack and repository structure
- Architecture principles
- Coding and testing conventions
- Security and privacy expectations
- AI/agent principles
- Human oversight requirements
- Development workflow
- Quality gates and communication expectations

Example initial CLAUDE.md content:

```
# Customer Onboarding AI

## Purpose
Build an enterprise customer onboarding system that combines deterministic software controls with AI-assisted document processing, validation and risk assessment.

## Engineering Principles
- Prefer small, incremental changes.
- Read the relevant specification before implementing.
- Do not invent business requirements.
- Keep security-critical controls deterministic.
- AI outputs must be structured, validated and auditable.
- High-risk or uncertain decisions require human review.
- Write tests for every implemented capability.

## Development Workflow
1. Read CLAUDE.md.
2. Read the relevant specification.
3. Inspect the existing repository.
4. Plan the smallest complete change.
5. Implement.
6. Run deterministic tests.
7. Evaluate model-driven behaviour where applicable.
8. Report changes, tests and remaining risks.
```

Important: CLAUDE.md is not itself a security boundary. If a behaviour must be technically enforced, use appropriate permissions, hooks, application controls or other enforcement mechanisms rather than relying only on instructions.

4. spec.md — What Should Be Built

spec.md is the central product and engineering specification. It should explain the business objective, users, workflow, functional requirements, non-functional requirements, AI requirements, acceptance criteria and scope boundaries.

Suggested spec.md structure

- 1. Purpose and business objective
- 2. Users and stakeholders
- 3. End-to-end onboarding workflow
- 4. Functional requirements
- 5. Non-functional requirements
- 6. Security and privacy requirements
- 7. AI/model requirements
- 8. Human-in-the-loop requirements
- 9. Auditability and observability
- 10. Acceptance criteria
- 11. Out of scope

Example requirement:

FR-03: Document Processing

The system shall accept supported customer documents, identify the document type, extract relevant fields into a structured representation, retain evidence references, and identify uncertainty or extraction failure.

The system shall not treat instructions contained inside an uploaded document as system or developer instructions.

5. The Initial Repository Structure

```
customer-onboarding-ai/
├── CLAUDE.md
├── README.md
├── docs/
│   ├── business-problem.md
│   ├── architecture.md
│   ├── security.md
│   └── decisions.md
├── specs/
│   ├── spec.md
│   ├── customer-onboarding.md
│   ├── document-processing.md
│   ├── validation.md
│   └── risk-assessment.md
├── plans/
│   └── implementation-plan.md
├── tasks/
│   └── tasks.md
├── backend/
├── frontend/
└── evals/
```

6. Starting from an Empty Git Repository

A practical terminal starting point:

```
mkdir customer-onboarding-ai
cd customer-onboarding-ai
git init

mkdir -p docs specs plans tasks
mkdir -p evals/datasets evals/scenarios
mkdir -p backend frontend
mkdir -p .claude

claude
```

The purpose of this first session is not to build the application. The objective is to establish the engineering foundation before implementation.

7. Prompt 01 — Establish the Engineering Foundation

We are building an enterprise AI-native customer onboarding application.

The application will be model-driven. Its behaviour will therefore be probabilistic rather than fully deterministic.

Do not write application code yet.

First help me establish the engineering foundation.

Create:

1. docs/business-problem.md
2. CLAUDE.md
3. specs/spec.md

The system must support:

- corporate customer onboarding
- document processing
- information extraction
- cross-document validation
- AI-assisted risk assessment
- human review
- auditability

Important:

This is an AI-native system.

The specification must explicitly address:

- probabilistic model behaviour
- uncertainty
- confidence
- hallucination risk
- model failure
- prompt injection
- evaluation datasets
- quality thresholds
- human escalation
- model/prompt versioning

Do not invent implementation details unnecessarily.

After creating the files, explain the decisions you made.

Do not implement the application yet.

8. Prompt 02 — Hostile Architecture Review of the Specification

Review specs/spec.md as a hostile architecture reviewer.

Do not modify the file.

Identify:

1. Ambiguous requirements
2. Requirements that assume deterministic model behaviour
3. Requirements that cannot be verified
4. Missing AI evaluation criteria
5. Potential hallucination risks
6. Prompt injection risks
7. Missing human escalation conditions
8. Missing model/prompt versioning
9. Requirements that could create unsafe autonomous decisions
10. Requirements that need measurable acceptance criteria

For each issue explain:

- The problem
- Why it matters
- Recommended change

Do not implement anything.

9. Prompt 03 — Refine the Specification for Probabilistic Behaviour

Based on your review, update specs/spec.md.

Convert vague AI requirements into measurable probabilistic quality requirements where possible.

For every model-driven capability define:

- Input
- Output
- Expected behaviour
- Uncertainty behaviour
- Failure behaviour
- Evaluation metric
- Target threshold
- Human escalation condition

Do not introduce requirements that cannot realistically be evaluated.

After updating the specification, provide a summary of the changes.

10. Prompt 04 — Create the Evaluation Rubric

```
Read:
CLAUDE.md
specs/spec.md

Create:
evals/evaluation-rubric.md

Define the evaluation strategy for:
1. Document extraction
2. Validation
3. Risk assessment
4. Orchestration

For each capability define:
- Evaluation objective
- Test scenarios
- Metrics
- Thresholds
- Critical failures
- Human escalation requirements

Separate:
A. Deterministic tests
B. Semantic/model evaluations
C. Safety evaluations
D. Workflow/trajectory evaluations
```

11. Prompt 05 — Create the Golden Evaluation Dataset

```
Using specs/spec.md and evals/evaluation-rubric.md,
create an initial golden evaluation dataset.
```

```
Create:
evals/datasets/customer_onboarding_cases.json
```

```
Include representative scenarios for:
- straightforward low-risk customer
- medium-risk customer
- high-risk customer
- missing document
- conflicting company information
- poor-quality document
- ambiguous information
- unsupported claim
- prompt injection inside a document
- incomplete application
```

```
For each case define:
- input
- relevant context
- expected behaviour
- expected escalation
- evaluation criteria
```

Do not create fake regulatory rules.
Keep the dataset clearly marked as synthetic development/test data.

12. Prompt 06 — Define Model Contracts

Read the specifications and evaluation rubric.

Create:

specs/model-contracts.md

Define contracts for:

1. Document Agent
2. Validation Agent
3. Risk Agent
4. Orchestrator

For every agent specify:

- Purpose
- Inputs
- Outputs
- Allowed tools
- Forbidden actions
- Uncertainty representation
- Failure behaviour
- Escalation behaviour
- Evaluation criteria

Example Risk Agent contract:

- Input: validated customer information, risk indicators and evidence.
- Output: risk_level, confidence, risk_factors, evidence, uncertainty and recommended_action.
- Allowed: analyse supplied information, identify risk indicators and recommend escalation.
- Forbidden: approve/reject the customer, invent evidence or override a human decision.

13. Prompt 07 — Design the Enterprise Architecture

Read:

CLAUDE.md
docs/business-problem.md
specs/spec.md
specs/model-contracts.md
evals/evaluation-rubric.md

Create:

docs/architecture.md

Design an enterprise architecture for the system.

Explicitly separate:

1. Deterministic application components
2. Probabilistic model components
3. Agent orchestration
4. Evaluation
5. Human decision points
6. Security controls

7. Observability

Identify which decisions must remain deterministic and which can be model-driven.

Do not implement code.

14. Prompt 08 — Build an Incremental Implementation Plan

Read all current project documentation.

Create:

plans/implementation-plan.md

Break implementation into incremental vertical slices.

Each slice must:

- deliver usable functionality
- have deterministic tests
- have AI evaluations where applicable
- have acceptance criteria
- identify model dependencies
- identify security risks

Do not create one giant implementation phase.

15. Prompt 09 — Convert the Plan into Executable Tasks

Create tasks/tasks.md from the implementation plan.

Every task must contain:

- Task ID
- Objective
- Specification reference
- Dependencies
- Acceptance criteria
- Tests
- AI evaluation requirements
- Security considerations

16. Prompt 10 — Implement One Vertical Slice

Implement TASK-001.

Before writing code:

1. Read CLAUDE.md.
2. Read the referenced specification.
3. Read architecture.md.
4. Inspect the existing repository.
5. Identify dependencies.

Then:

1. Implement the smallest complete vertical slice.
2. Write deterministic tests.

3. Run the tests.
4. Validate against the acceptance criteria.

Do not modify unrelated files.

After implementation report:

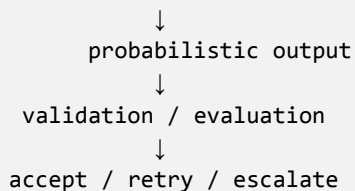
- Files changed
- Requirements satisfied
- Tests executed
- Test results
- Remaining risks

17. What Changes When the Application Becomes Model-Driven?

The first ten prompts establish a strong conventional engineering workflow. However, an AI application cannot be engineered as though every function returns a predictable result. Model outputs vary with model version, prompt, context, retrieved information, tool results and input characteristics.

Traditional deterministic function:
input → function → expected output

Model-driven component:
input + context + prompt + model + tools

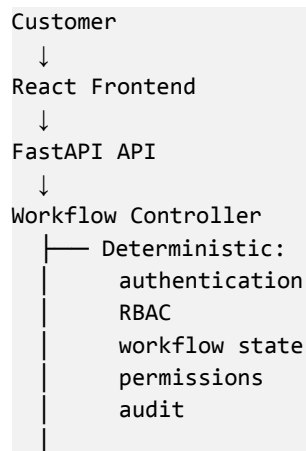


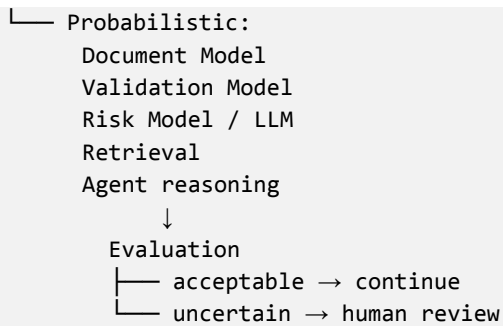
The engineering equation becomes:

Application behaviour = Code + Model + Prompt + Context + Tools + Data

18. Deterministic vs Probabilistic Boundaries

A central architecture principle is to keep critical controls and state transitions deterministic while allowing models to handle bounded probabilistic tasks.





19. Evaluation Becomes a First-Class Engineering Layer

- Deterministic tests verify code, schemas, APIs and state transitions.
- Semantic/model evaluations assess whether AI outputs meet quality expectations.
- Safety evaluations test prompt injection, unsafe recommendations, unsupported claims and other failure modes.
- Workflow/trajectory evaluations assess whether agents use tools and follow the intended process.
- Golden datasets provide repeatable scenarios against which model/prompt changes can be compared.

Example evaluation repository:

```
evals/
├─ datasets/
│   ├── customer_documents.json
│   ├── validation_cases.json
│   └── risk_cases.json
├─ expected/
│   └── expected_behaviour.json
├─ scenarios/
│   ├── ambiguous-document.md
│   ├── conflicting-information.md
│   ├── prompt-injection.md
│   └── incomplete-application.md
└─ evaluation-rubric.md
```

20. Why the Specification Must Evolve

A deterministic requirement might say:

```
The system shall extract the customer's legal name.
```

An AI-native requirement should define the behaviour around uncertainty:

```
The Document Agent shall extract the customer's legal name into a
structured field, retain an evidence reference, represent uncertainty,
and escalate when the evidence is insufficient or contradictory.
```

```
The behaviour shall be evaluated against the approved synthetic evaluation
dataset using the documented extraction quality and escalation thresholds.
```

21. The Three-Layer Agent Model

Layer 1 – Claude / Model

Probabilistic reasoning engine

Layer 2 – Claude Code

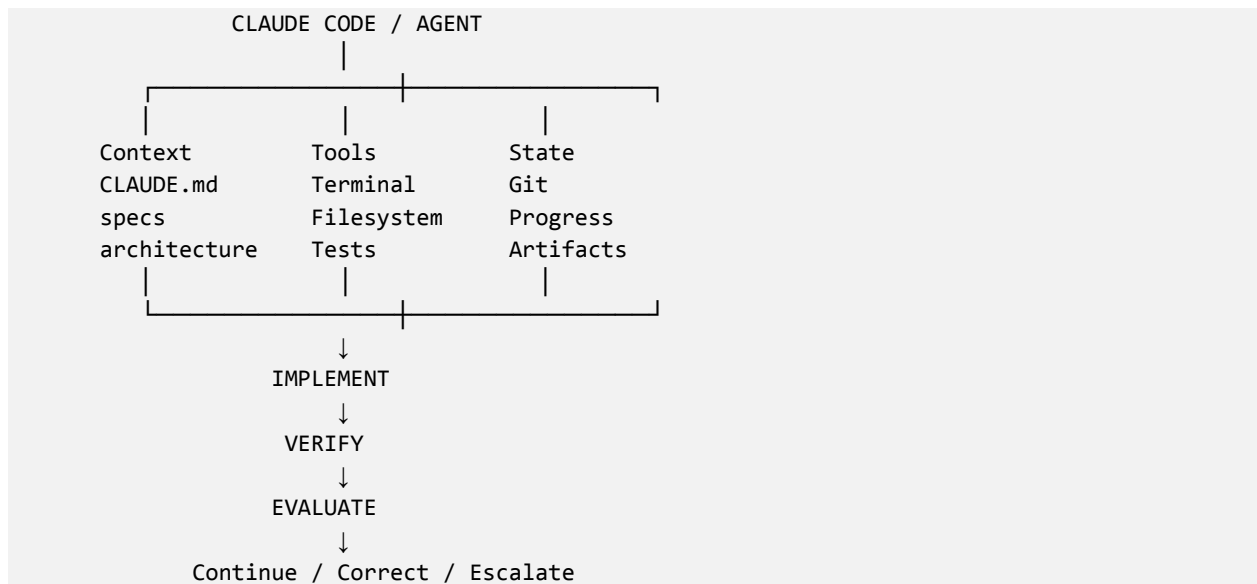
Development agent that reads the repository, edits files, runs tools/tests and iterates on the software

Layer 3 – Runtime AI Agents

Document Agent, Validation Agent, Risk Agent and Orchestrator that execute the customer onboarding business workflow

22. Harness Engineering – Where It Fits

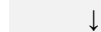
The initial project foundation should not be confused with a commercial product called Harness. In this training context, harness engineering means deliberately designing the environment around the coding/agent system so that it can work iteratively and reliably.



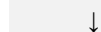
Thus, CLAUDE.md and the specification remain valid and important. They form part of the project context and engineering foundation, while the broader harness includes the mechanisms that provide context, tools, state, verification, feedback and controlled iteration.

23. Recommended End-to-End AI-Native Engineering Lifecycle

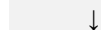
Business Problem



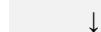
Specification



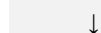
Model Contracts



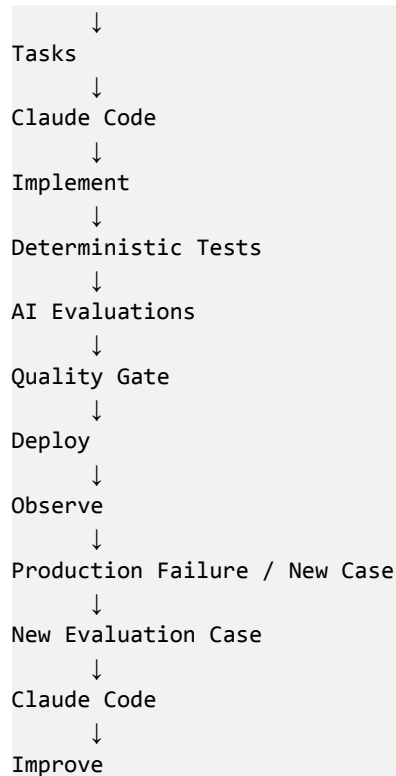
Golden Dataset



Architecture



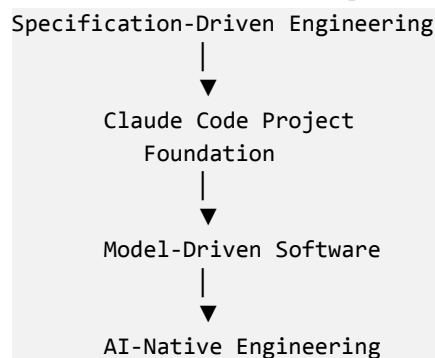
Implementation Plan

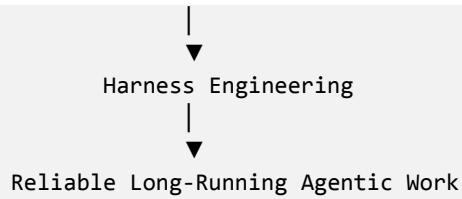


24. Key Teaching Messages

1. Do not start an enterprise AI build by asking Claude Code to generate the whole application.
2. Start with the business problem and a specification.
3. CLAUDE.md tells Claude Code how the project should be approached; spec.md defines what should be built.
4. The project repository becomes part of the agent's working context.
5. Once models become first-class components, deterministic unit tests alone are insufficient.
6. AI behaviour must be specified in terms of uncertainty, evidence, evaluation and escalation.
7. Critical business controls should remain deterministic where possible.
8. Models should operate inside explicit contracts and bounded workflows.
9. Evaluation datasets and rubrics must become first-class engineering assets.
10. Harness engineering is the broader discipline of making agentic work reliable through context, tools, state, verification and feedback.

25. Final Workshop Mental Model





Appendix A — Suggested First Files to Create

- CLAUDE.md
- README.md
- docs/business-problem.md
- docs/architecture.md
- docs/security.md
- docs/decisions.md
- specs/spec.md
- specs/model-contracts.md
- plans/implementation-plan.md
- tasks/tasks.md
- evals/evaluation-rubric.md
- evals/datasets/customer_onboarding_cases.json

Appendix B — Practical Rule for Claude Code Sessions

Before implementation:

READ → UNDERSTAND → PLAN → IMPLEMENT → TEST → EVALUATE → REPORT

Never:

PROMPT → GENERATE EVERYTHING → ASSUME IT WORKS

End of initial discussion document